

PROJET GL SUIVI 2

AKRAM AHMED
IKHLAS
THIBAUT WIDAD
GR 9 – 51

ENCADRÉ PAR : GABRIELA NICOLE GONZALEZ SAEZ



Planning

ID	Nom de la tâche	Durée	Prédécesseurs	Ressources
2	PHASE 2 : SANS OBJET & EXTENSION	5 j		
	Complétion Grammaire A (Sans objet)	1.5 j		ISI, MMIS B
	Vérification Contextuelle B (Sans objet)	3 j		ISI, MMIS B
	Génération de Code C (Arithm/Control)	4 j		MMIS A
	Suites de Tests (Syntaxe & Sémantique)	4.5 j		MMIS C, IF
	Étude & Début de l'Extension	3 j	1.2, 2.1	MMIS B, IF
	Intégration globale & Suivi 2	0.5 j	2.2, 2.3, 2.4	Tout le monde



Avancement étape B et C



étape C

État d'avancement - Deca "Sans Objet"

- Infrastructure de base :
 - Intégration du RegisterManager dans DecacCompiler.
 - Squelette Program.java fonctionnel (Entête, Corps, Fin).
- Fonctionnalités :
 - Arithmétique : Expressions binaires (+, -, *, /, %) et unaires.
 - Mémoire : Déclaration et initialisation de variables globales (adressage x(GB)).
 - E/S : Instructions print, println (Entiers, Flottants, Chaînes). (pas encore implemente)
 - Contrôle : Structures IfThenElse et While (pas encore implemente).
- Gestion de la Pile :
 - Calcul de la taille nécessaire (TSTO).
 - Détection de débordement .

Architecture Technique & Choix d'Implémentation

- Gestion des Registres (RegisterManager) :
 - Pool de registres libres (R2 à R15).
 - Stratégie d'allocation naïve (prochain libre) avec libération immédiate après usage temporaire.
 - Limitation actuelle : Pas de gestion du "Spill" (erreur si > 14 registres nécessaires simultanément).
- Pattern de Génération (codeGen) :
 - Instructions (codegenInst) : Ne retournent rien, écrivent directement dans le buffer (ex: WINT, BRA).
 - Expressions (codegenExpr) : Prennent un GPRegister cible en paramètre.
 - Exemple : `Plus.codeGenExpr(compiler, R2)` calcule l'opérande gauche dans R2, alloue R3 pour la droite, fait `ADD R3, R2`, puis libère R3.

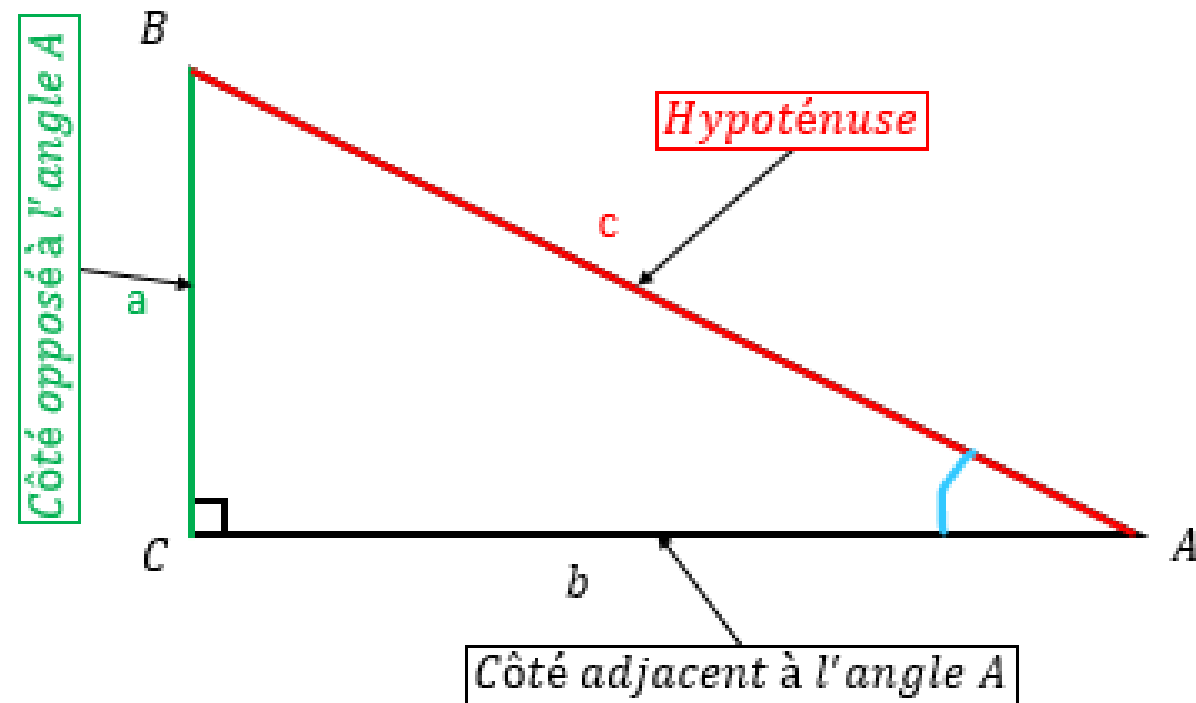
Validation & Prochaines Étapes

- Tests "bout-en-bout" : Compilation .deca → .ass → Exécution ima.
- Validation manuelle sur des cas critiques :
- Priorité des opérateurs.
- Imbrication de If/Else et While.
- Débordement de pile (testé avec un petit StackSize).
- Passe 1 : Construction de la Table des Méthodes (VTable) pour chaque classe.
- Passe 2 : Génération du constructeur et de l'initialisation des champs (init.Class).
- Appels de méthodes : Gestion de TSTO dynamique (sauvegarde registres) et adressage x(LB).
- Opérateur New : Allocation dans le Tas.



Avancement tests

Extension TRIGO





Architecture de l'extension

1 - 1 + epsilon != 1+epsilon - 1

Fournir une classe **Math** dans la bibliothèque standard de **Deca**, permettant d'utiliser des fonctions mathématiques avancées

- float sin(float f)
- float cos(float f)
- float asin(float f)
- float atan(float f)
- float ulp(float f)

Précision des calculs

- Les fonctions doivent retourner la meilleure approximation possible permise.
- Les résultats doivent être dans le codomaine mathématique exact de la fonction (ex. asin doit retourner une valeur dans $[-\pi/2, \pi/2]$)

Gestion des erreurs

- Un appel à asin avec un argument hors de $[-1.0, 1.0]$ doit provoquer une erreur d'exécution
- Les débordements flottants doivent aussi être gérés

Idees d'implémentation

- Utiliser l'instruction FMA (Fused Multiply-Add) pour limiter les erreurs d'arrondi.
- Préférer les littéraux hexadécimaux pour représenter exactement les constantes flottantes.
- Valider les résultats par des comparaisons graphiques ou avec des références externes.

Pour ulp(x) : distance entre x et le nombre flottant immédiatement supérieur dans la représentation IEEE 754.

Implémentation : extraire l'exposant avec des opérations bit-à-bit (en assembleur IMA).

Approche 1: Table précalculée + Interpolation



- Précision homogène → Facile à valider
- Performance garantie (accès mémoire + calcul simple)

Interpolation linéaire

$$f(x) \approx f(x_0) + (x - x_0) \cdot \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

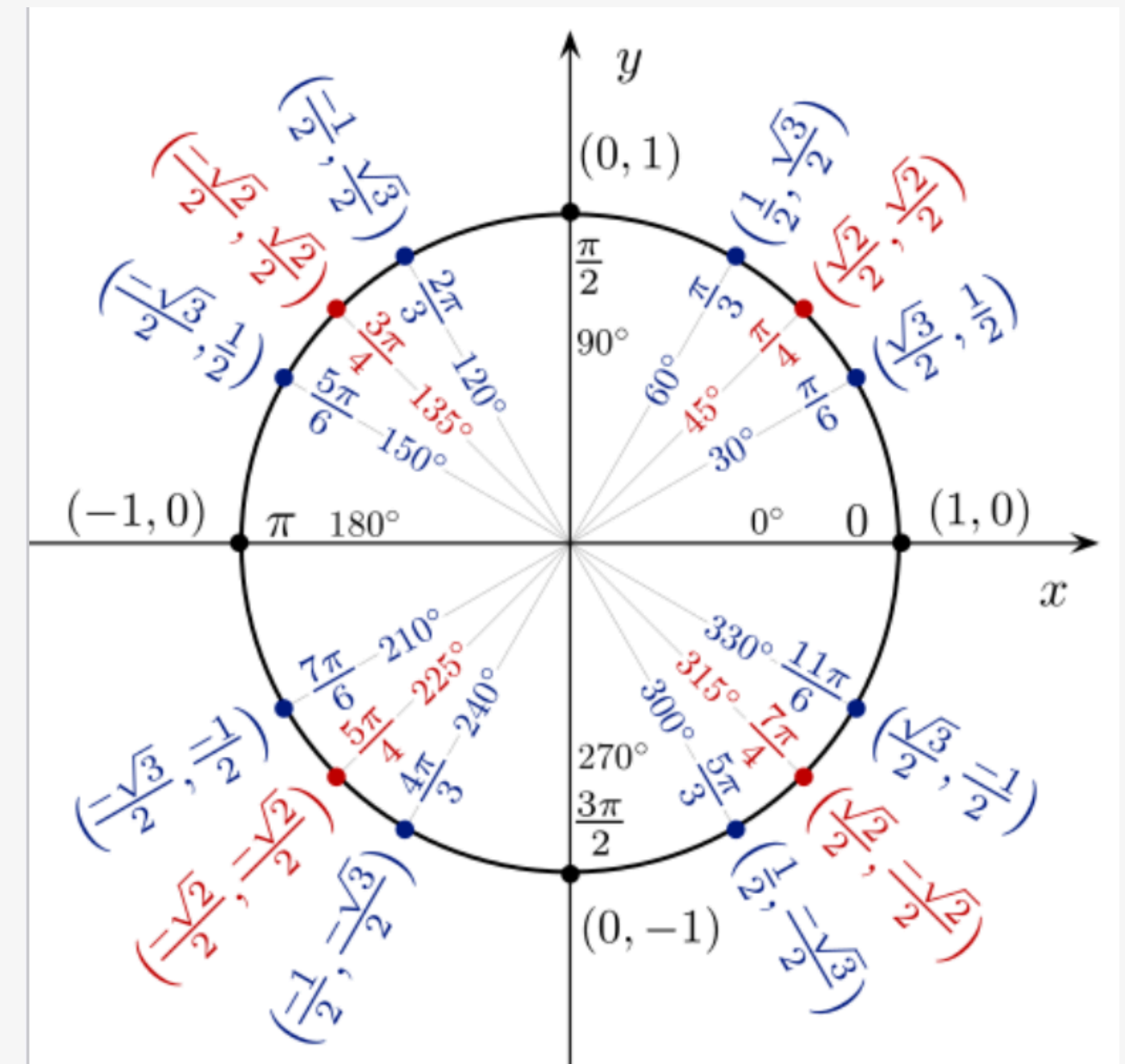


- Mémoire élevée pour 512-1024 points en float)
- Précision limitée par taille table
- Interpolation imparfaite près des extrêmes

$$L(X) = \sum_{j=0}^n y_j \left(\prod_{i=0, i \neq j}^n \frac{X - x_i}{x_j - x_i} \right)$$

Intrepolation d'Hermite

$$p(t) = h_{00}(t)p_0 + h_{10}(t)m_0 \cdot (x_1 - x_0) + h_{01}(t)p_1 + h_{11}(t)m_1 \cdot (x_1 - x_0).$$





Approche2 : Series de Taylor/Approximation polynomiale



- Pas de mémoire (tout calculé)
- Précision théorique infinie
- Meilleur pour très petit x (peu de termes)

$$T = f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \frac{f^{(3)}(a)}{3!}(x-a)^3 + \dots + \frac{f^{(n)}(a)}{n!}(x-a)^n$$

$$T(f, a, x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n$$



- Catastrophe pour $x > 1$ (convergence lente)
- Coût calcul (10-15 termes pour bonne précision)
- Rayon convergence limité (nécessite réduction)
- depend de la reduction

Pour sin(x) et cos(x)

- Série de Taylor valide pour tout x (rayon infini).
- **MAIS** : converge très lentement si $|x|$ n'est pas petit.

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \dots = \sum_{k=0}^n \frac{(-1)^k x^{2k}}{(2k)!}$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots = \sum_{k=0}^n \frac{(-1)^k x^{2k+1}}{(2k+1)!}$$

$$\arcsin(x) = \sum_{n=0}^{\infty} \frac{1}{4^n} \binom{2n}{n} \frac{x^{2n+1}}{2n+1}$$

$$\arccos(x) = \frac{\pi}{2} - \sum_{n=0}^{\infty} \frac{1}{4^n} \binom{2n}{n} \frac{x^{2n+1}}{2n+1}$$

$$\arctan(x) = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{2n+1}$$

Pour asin(x) et atan(x)

!!!! Série de Taylor valide seulement pour $|x| < 1$.
(Convergence très lente près de $|x| = 1$)

Approche2 : Optimisation

Idee : Réduction d'intervalle

Réduire l'angle dans $[-\pi/4, \pi/4]$ avant d'appliquer une approximation.
Utiliser une approximation polynomiale optimisée sur cet intervalle.
(Polynomes minimax , Chebychev ... plus efficace que Taylor)

Idee : Identités Trigonométriques

Pour $\text{atan}(x)$: si $|x| > 1$, on utilise $\text{atan}(x) = \pi/2 - \text{atan}(1/x)$.

Pour $\text{asin}(x)$: on le transforme en $\text{atan}(x / \sqrt{1 - x^2})$

Approche3: Calcul Hybride

$$\epsilon = 2^{-12} \approx 2.44 \times 10^{-4}$$

$x < 10^{-4}$

Taylor avec $n = 4$ ou 5

ou directement $\sin(x) \sim x$;
et $\cos(x) \sim 1$; $\arcsin(x) \sim x$; $\arctan(x) \sim$

< 0.5 ULP

$x < 0.1$

$$\sin(\text{base} + \text{incr}) = \cos(\text{incr}) \cdot \sin(\text{base}) + \sin(\text{incr}) \cdot \cos(\text{base})$$
$$\sin(\text{base} + \text{incr}) = \sin(\text{base}) + \sin(\text{incr}) * (-\tan(\text{incr}/2) * \sin(\text{base}) + \cos(\text{base}))$$

$$\cos(\text{base} + \text{incr}) = \cos(\text{incr}) \cdot \cos(\text{base}) - \sin(\text{incr}) \cdot \sin(\text{base})$$
$$\cos(\text{base} + \text{incr}) = \cos(\text{base}) - \sin(\text{incr}) \cdot (\tan(\text{incr}/2) \cdot \cos(\text{base}) + \sin(\text{base}))$$

$x \sim \pi/2$

$\text{delta} = x - \pi/2$ et donc
 $\sin(x) = \cos(\text{delta})$ et $\cos(x) = -\sin(\text{delta})$

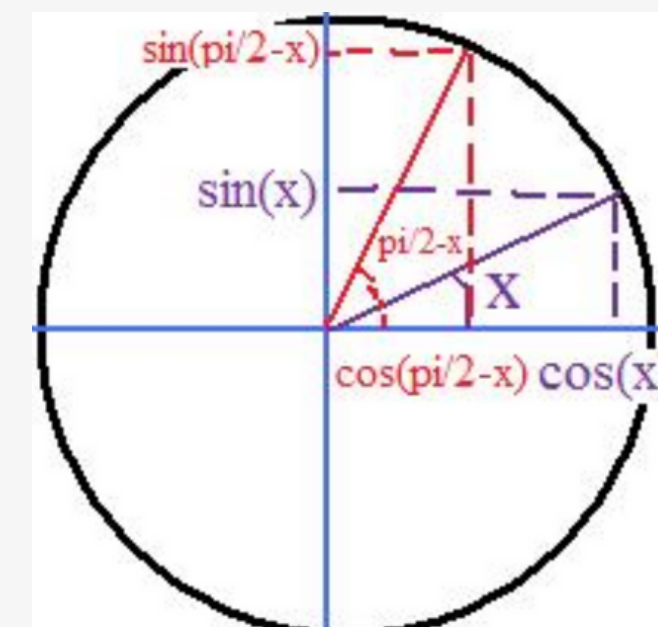
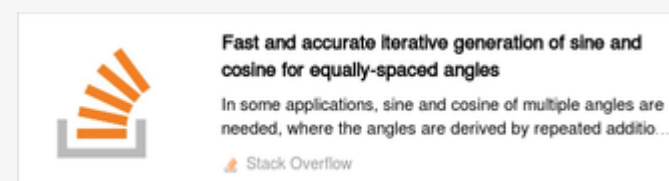
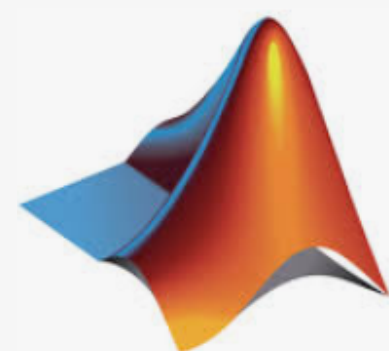


Table de 512/1024 valeurs precaculuse

else

Algorithme CORDIC





Algorithme CORDIC

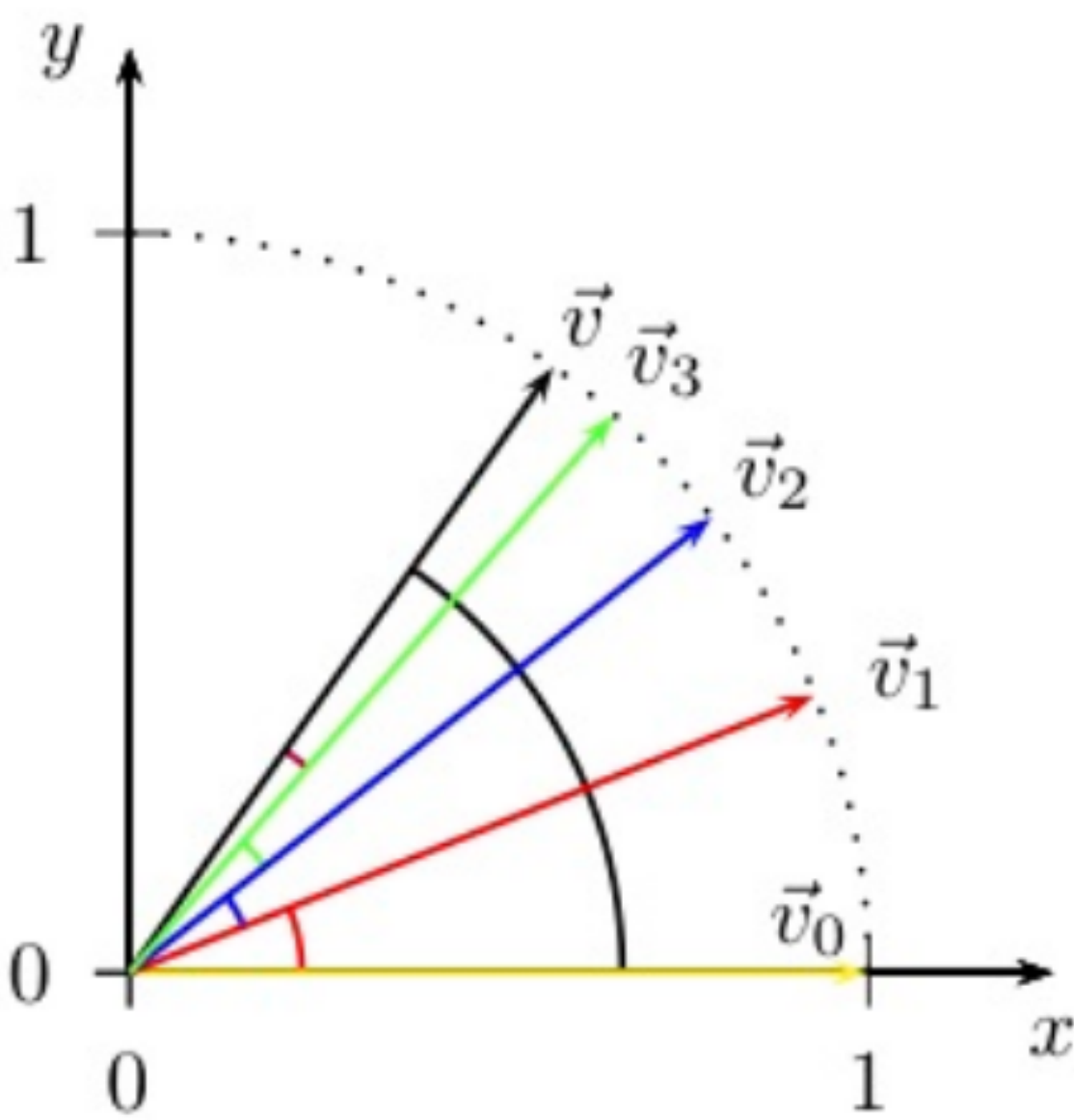
$$\sum_{i=0}^n \theta_i = \theta \quad \tan(|\theta_i|) = 2^{-i}$$

$$\vec{v}_{i+1} = \begin{bmatrix} x_{i+1} \\ y_{i+1} \end{bmatrix} = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i) \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix}$$

$$\vec{v}_{i+1}^* = \begin{bmatrix} x_{i+1}^* \\ y_{i+1}^* \end{bmatrix} = \begin{bmatrix} 1 & -\tan(\theta_i) \\ \tan(\theta_i) & 1 \end{bmatrix} \begin{bmatrix} x_i^* \\ y_i^* \end{bmatrix} = \begin{bmatrix} x_i^* - \tan(\theta_i) \cdot y_i^* \\ y_i^* + \tan(\theta_i) \cdot x_i^* \end{bmatrix}$$

$$x_{i+1}^* = x_i^* - \sigma_i \cdot 2^{-i} \cdot y_i^* ,$$

$$y_{i+1}^* = y_i^* + \sigma_i \cdot 2^{-i} \cdot x_i^* .$$



$\tan(\theta_i)$	$ \theta_i $	σ_i	$\sum \sigma_i \theta_i $
$2^0 = 1$	$0.785398 \equiv 45^\circ$	+	45°
$2^{-1} = 0.5$	$0.4636 \equiv 26.5651^\circ$	+	71.5651°
$2^{-2} = 0.25$	$0.2449 \equiv 14.0362^\circ$	-	57.5288°
$2^{-3} = 0.125$	$0.124 \equiv 7.1250^\circ$	-	50.4038°
$2^{-4} = 0.0625$	$0.0624 \equiv 3.5763^\circ$	+	53.9801°

Pourquoi est ce Cordic peut fonctionner mieux?

standard IEEE 754 simple précision (32 bits)

- 1 bit de signe
- 8 bits d'exposant (biaisé de +127)
- 23 bits de mantisse

$$n = 40$$

$$2^{-40} = 9.094947017729282e-13$$

Hex: 0x2B800000

0 01010111 000000000000000000000000

- **Opérations de base :**
décalages (>>) et additions/soustractions.

Gamme d'entrée limitée directe

Pour sin/cos : réduction d'angle nécessaire
pour ramener dans $[-\pi/2, \pi/2]$.

Cette réduction doit être exacte pour
respecter IEEE 754

Théorème de Weierstrass (approximation par des polynômes)

Théorème : Toute fonction continue
sur un segment, à valeurs dans $\mathbb{C}$, est
limite uniforme d'une suite de
fonctions polynomiales sur ce segment.

sin : Polynôme de degré 13 (minimax/Chebyshev)
 $x, x^3, x^5 \dots x^{13}$

cos : Polynôme de degré 12 (minimax/Chebyshev)
 $1, x^2, \dots x^{12}$

